Static Website Hosting on AWS

Using S3 + CloudFront with Terraform

Project Documentation & Step-by-Step Guide

Author	Ritik
Project	cyber-security-operations-interactive-engine
Team	DevOps Team
Environment	dev
AWS Region	us-east-1
Date	22-05-2026
Terraform Version	~> 6.43.0 (AWS Provider)

---

1. Project Overview

This project provisions a fully automated, production-grade static website hosting infrastructure on AWS using Terraform (Infrastructure as Code). The architecture uses Amazon S3 for object storage and Amazon CloudFront as a CDN, ensuring the website is served securely over HTTPS with low latency globally.

Key highlights:

- Private S3 bucket — no public access; all traffic routed via CloudFront
- Origin Access Control (OAC) — modern, secure method for CloudFront-to-S3 authentication
- Automatic file uploads — all files in the www/ directory uploaded via Terraform
- HTTPS enforced — HTTP requests redirected to HTTPS
- Infrastructure as Code — 100% reproducible with a single terraform apply

---

2. Project File Structure

The project directory is organized as follows:

```
Static-website-s3/
  .terraform/              # Terraform internal files & provider cache
  www/                     # Static website files (HTML, CSS, JS, images)
  .gitignore               # Git ignore rules
  .terraform.lock.hcl      # Provider version lock file
  backend.tf               # Terraform remote state configuration
  locals.tf                # Local variables (content-type mappings)
  main.tf                  # Core infrastructure resources
  outputs.tf               # Output values (URLs, IDs)
  providers.tf             # AWS provider configuration
  terraform.tfstate        # Current state file
  terraform.tfstate.backup # Backup of previous state
  variables.tf             # Input variables
```

---

3. Prerequisites

Before starting, the following tools and credentials are required:

- Terraform CLI installed (v1.0+)
- AWS CLI configured with valid credentials (aws configure)
- An AWS account with permissions for: S3, CloudFront, IAM policies
- A local www/ directory containing the static website files (index.html, CSS, JS, etc.)

---

4. Step-by-Step Implementation Procedure

Step 1 — Initialize the Project Directory

A new project folder was created and navigated into via PowerShell:

```
mkdir Static-website-s3
cd Static-website-s3
```

This is the working directory for all Terraform configuration files.

Step 2 — Configure the AWS Provider (providers.tf)

The providers.tf file was created to declare the Terraform and AWS provider requirements. The AWS provider version was pinned to ~> 6.43.0 to avoid breaking changes from future updates.

```
terraform {
  required_providers {
    aws = {
```

```
      source  = "hashicorp/aws"
      version = "~> 6.43.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

Step 3 — Define Variables (variables.tf)

Input variables were defined to make the configuration reusable. The bucket name and resource tags are parameterized, making it easy to deploy to different environments or names by simply changing variable values.

```
variable "bucket_name" {
  default = "ritik-s3-website-hosting-21-05-26"
}

variable "tags" {
  type = map(string)
  default = {
    Name        = "ritik-s3-website-hosting-21-05-26"
    Environment = "dev"
    owner       = "Ritik sharma"
    purpose     = "Static website hosting"
    project     = "cyber-security-operations-interactive-engine"
    team        = "DevOps Team"
  }
}
```

Step 4 — Define Local Values (locals.tf)

A locals.tf file was created to store a content-type lookup map. This is used when uploading files to S3 to ensure each file is served with the correct MIME type, so browsers render HTML, CSS, and JS correctly.

```
locals {
  content_type = {
    html = "text/html"
    css  = "text/css"
    js   = "application/javascript"
    png  = "image/png"
    jpg  = "image/jpeg"
    jpeg = "image/jpeg"
```

```
    gif  = "image/gif"
  }
}
```

Step 5 — Build Core Infrastructure (main.tf)

This is the most important file. It was built step by step, with each resource serving a specific purpose in the architecture.

5a. Create the S3 Bucket

An S3 bucket was provisioned using the variable-defined bucket name and tags.

```
resource "aws_s3_bucket" "firstbucket" {
  bucket = var.bucket_name
  tags   = var.tags
}
```

5b. Block All Public Access

All four public access block settings were enabled to ensure the bucket remains completely private. Direct public access to the bucket is not allowed — access is only via CloudFront.

```
resource "aws_s3_bucket_public_access_block" "s3_public_access_block" {
  bucket                  = aws_s3_bucket.firstbucket.id
  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}
```

5c. Create Origin Access Control (OAC)

An OAC was created to securely authenticate CloudFront's requests to S3. OAC uses AWS Signature Version 4 (sigv4) signing, which is the recommended modern approach (replacing the older Origin Access Identity / OAI).

```
resource "aws_cloudfront_origin_access_control" "s3_oac" {
  name                              = "s3-oac"
  description                       = "Origin Access Control for S3 bucket"
  origin_access_control_origin_type = "s3"
  signing_behavior                  = "always"
  signing_protocol                  = "sigv4"
}
```

5d. Attach a Bucket Policy for CloudFront

A bucket policy was attached that allows only CloudFront (identified by the cloudfront.amazonaws.com service principal) to perform s3:GetObject on all objects in the bucket. This ensures zero-trust: no one else can access the files directly.

```
resource "aws_s3_bucket_policy" "website_bucket_policy" {
  bucket     = aws_s3_bucket.firstbucket.id
  depends_on = [aws_s3_bucket_public_access_block.s3_public_access_block]
  policy = jsonencode({
    Statement = [{
      Effect    = "Allow"
      Principal = { Service = "cloudfront.amazonaws.com" }
      Action    = "s3:GetObject"
      Resource  = "${aws_s3_bucket.firstbucket.arn}/*"
    }]
  })
}
```

5e. Upload Website Files to S3

All files from the local www/ directory are uploaded automatically using the fileset() function combined with for_each. Each file is uploaded with the correct content type determined by its extension using the locals content_type map. The etag ensures Terraform detects file changes and re-uploads only when files are modified.

```
resource "aws_s3_object" "website_files" {
  for_each     = fileset("${path.module}/www", "**/*")
  bucket       = aws_s3_bucket.firstbucket.id
  key          = each.value
  source       = "${path.module}/www/${each.value}"
  content_type = lookup(local.content_type, split(".",
each.value)[length(split(".", each.value)) - 1], "application/octet-stream")
  etag         = filemd5("${path.module}/www/${each.value}")
}
```

5f. Create CloudFront Distribution

A CloudFront distribution was created in front of the S3 bucket. Key configuration decisions made:

- Origin: Points to the S3 regional domain name with the OAC attached
- Default root object: index.html — the homepage served at the root URL
- HTTPS enforced: viewer_protocol_policy = redirect-to-https
- IPv6 enabled: is_ipv6_enabled = true
- Geo-restriction: none — globally accessible
- Caching: TTL configured as min=0, default=3600, max=86400 seconds
- Compression: compress = true — enables gzip/brotli for faster load times
- Certificate: cloudfront_default_certificate = true (*.cloudfront.net domain)

Step 6 — Define Outputs (outputs.tf)

Output values were defined so key resource identifiers and URLs are displayed after terraform apply. This makes it easy to quickly access the deployed website without navigating the AWS Console.

```
output "s3_bucket_name"                    { value = aws_s3_bucket.firstbucket.id }
output "cloudfront_distribution_id"        { value =
aws_cloudfront_distribution.website_distribution.id }
output "cloudfront_distribution_domain"    { value =
aws_cloudfront_distribution.website_distribution.domain_name }
output "website_url"                       { value =
"https://${aws_cloudfront_distribution.website_distribution.domain_name}" }
```

Step 7 — Initialize Terraform

Terraform was initialized to download the AWS provider plugin and set up the backend:

```
terraform init
```

This created the .terraform/ directory and generated the .terraform.lock.hcl file locking the provider versions.

Step 8 — Validate & Plan

The configuration was validated for syntax errors, then a plan was reviewed to see what resources would be created:

```
terraform validate
terraform plan
```

The plan output showed all resources to be created: S3 bucket, public access block, OAC, bucket policy, S3 objects (website files), and the CloudFront distribution.

Step 9 — Apply the Configuration

The infrastructure was deployed by running:

```
terraform apply
```

Terraform provisioned all resources in the correct dependency order. After completion, the outputs were displayed — including the live website URL on the CloudFront domain.

Step 10 — Verify the Deployment

The website was verified by opening the output website_url in a browser:

```
https://<distribution-id>.cloudfront.net
```

The static website loaded successfully over HTTPS. Direct access to the S3 bucket URL was confirmed to be blocked (Access Denied), confirming the private + OAC setup works correctly.

5. Architecture Overview

The deployed architecture follows this request flow:

```
User Browser  -->  CloudFront Distribution  -->  S3 Bucket (Private)
```

Detailed flow:

- User sends HTTPS request to the CloudFront domain
- CloudFront checks its edge cache — cache hit returns immediately
- Cache miss: CloudFront authenticates to S3 using the OAC (sigv4 signature)
- S3 validates the request against the bucket policy (allows cloudfront.amazonaws.com)
- S3 returns the requested file to CloudFront
- CloudFront caches the file and serves it to the user

6. Key Design Decisions & Notes

Decision	Reasoning
OAC over OAI	OAC is the modern AWS-recommended approach. OAI is legacy and being deprecated.
Private S3 bucket	Zero direct public exposure; all traffic forced through CloudFront for security & caching.
fileset() + for_each	Dynamically discovers and uploads all files in www/ without hardcoding filenames.
etag = filemd5()	Ensures Terraform detects file content changes and re-uploads modified files only.
depends_on on distribution	Prevents race condition where CloudFront is created before the bucket policy is attached.
redirect-to-https	Enforces encrypted transport; HTTP users are automatically redirected to HTTPS.
compress = true	Enables automatic gzip/brotli compression, reducing file sizes and improving load speed.
Duplicate Version key in policy	A minor bug exists in main.tf — the policy block has Version declared twice. Harmless (last value wins) but should be cleaned up.

7. Terraform Commands Reference

Initialize project and download providers:

```
terraform init
```

Validate configuration syntax:

```
terraform validate
```

Preview changes before applying:

```
terraform plan
```

Deploy infrastructure:

```
terraform apply
```

Destroy all created resources:

```
terraform destroy
```

Show current state:

```
terraform show
```

---

8. Project Summary

Summary of What Was Done — Step by Step

1. Created the Terraform project directory and set up all configuration files: providers.tf, variables.tf, locals.tf, main.tf, outputs.tf.
2. Configured the AWS provider for the us-east-1 region, pinned to version ~> 6.43.0 for stability.
3. Defined reusable input variables for the bucket name and resource tags (environment, team, project, owner).
4. Set up a local content-type map to automatically assign correct MIME types to uploaded files (HTML, CSS, JS, images).
5. Provisioned a private S3 bucket with all four public access block settings enabled — no direct public internet access allowed.
6. Created a CloudFront Origin Access Control (OAC) using sigv4 signing — the modern, secure way for CloudFront to authenticate with S3.
7. Attached an S3 bucket policy allowing only the CloudFront service principal to read objects — enforcing zero-trust access.

8. Automated the upload of all website files from the local www/ directory to S3 using fileset() and for_each, with correct content types and MD5-based change detection.
9. Created a CloudFront distribution pointing at the private S3 bucket, with HTTPS enforcement, IPv6 support, caching, gzip compression, and a global edge network.
10. Defined output values to display the S3 bucket name, CloudFront distribution ID, domain name, and the final live website URL after deployment.
11. Ran terraform init, terraform validate, terraform plan, and terraform apply to deploy the full infrastructure.
12. Verified the website loads correctly over HTTPS via the CloudFront URL, and confirmed direct S3 access is blocked.

Result: A secure, globally distributed, HTTPS-only static website hosted on AWS — fully provisioned and reproducible using Terraform Infrastructure as Code.